



数据库系统

第八章 数据库编程

胡金鹏



8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

8.5 OLE DB

8.6 JDBC编程

8.7 小结



SQL的使用方式

1、**交互式**：在终端上每输入一条SQL语句，系统立即执行，然后等待用户输入下一条语句。

2、**嵌入式**（嵌入到某种主语言中使用）：

宿主语言负责：运算、处理、流程控制等

SQL负责：数据库操作

- 为什么要引入嵌入式SQL?

- SQL语言是非过程性语言

有些操作对于交互式SQL是**不可能的任务**。

- 事务处理应用需要高级语言

非声明性动作:用户交互、图形化显示数据



8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不使用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.6 小结

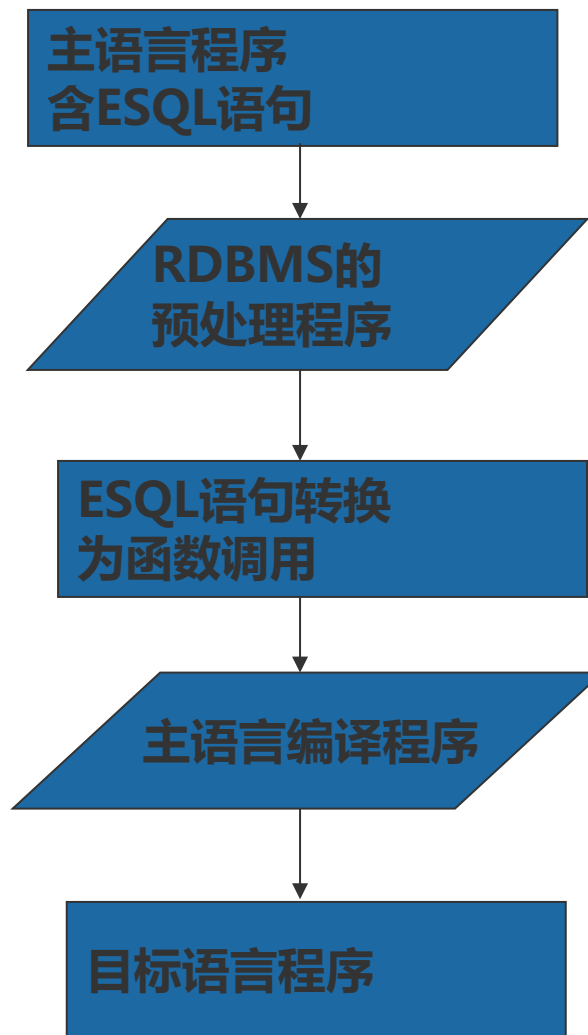


- 主语言

- 嵌入式SQL是将SQL语句嵌入程序设计语言中，被嵌入的程序设计语言，如C、C++、Java，称为宿主语言，简称主语言。

- 处理过程

- 预编译方法



1. 由RDBMS的预处理程序对源程序进行扫描，识别出SQL语句
2. 把它们转换成主语言调用语句，以使主语言编译程序能识别它
3. 最后由主语言的编译程序将整个源程序编译成目标码。

ESQL基本处理过程



■ 嵌入式SQL的使用规定

1、在程序中要区分SQL语句与宿主语言语句

嵌入的SQL语句以**EXEC SQL**开始，以分号(;) 或**END_EXEC**结束（根据具体语言而定）。

EXEC SQL <语句> **END_EXEC**

【例】

```
EXEC SQL        delete from s  
                 where sno= ' s10' ;
```



嵌入式SQL的使用规定

2、允许嵌入的SQL语句引用宿主语言的程序变量（共享变量），但有以下规定：

(1) 宿主变量出现于SQL语句中时，前面加(:)以区别数据库变量（列名）

宿主变量可出现的地方：SQL的数据操纵语句中可出现**常数**的任何地方，select等语句的**into**字句中。

示例：
*EXEC SQL select SNAME ,AGE
 into :stu_name , :age
 from s
 where SNO = :input_no ;*



嵌入式SQL的使用规定

(2)共享变量的用法：先由宿主语言的程序定义，并用SQL的**DECLARE**语句说明

```
EXEC SQL BEGIN DECLARE SECTION
```

```
    int    stu_no;
```

```
    char  stu_name[30];
```

```
    int    age;
```

```
    char  SQLSTATE[6]
```

```
EXEC SQL END DECLARE SECTION
```

```
EXEC SQL select  sname , age
```

```
           into   :stu_name ,:age      from    s
```

```
           where  sno= :stu_no ;
```




嵌入式SQL语句与主语言之间的通信

将SQL嵌入到高级语言中混合编程，程序中会含有两种不同计算模型的语句

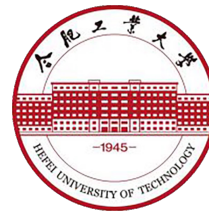
SQL语句

- 描述性的面向集合的语句
- 负责操纵数据库

高级语言语句

- 过程性的面向记录的语句
- 负责控制程序流程

它们之间应该如何通信？



8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不使用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.6 小结



- 数据库工作单元与源程序工作单元之间的通信：

1. SQL通信区 (SQL Communication Area , 简称SQLCA)

- 向主语言传递SQL语句的执行状态信息
- 使主语言能够据此控制程序流程

2. 主变量(host variable)

- 主语言向SQL语句提供参数
- 将SQL语句查询数据库的结果交主语言进一步处理

3. 游标 (cursor)

- 将SQL语句查询数据库的结果交主语言进一步处理
- 解决集合性操作语言与过程性操作语言的不匹配



SQL通信区

- **SQLCA : SQL Communication Area**
SQLCA是一个数据结构
- **SQLCA的用途**
 - **SQL语句执行后，RDBMS反馈给应用程序信息**
 - ✓ **描述系统当前工作状态**
 - ✓ **描述运行环境**
 - **这些信息将送到SQL通信区SQLCA中**
 - **应用程序从SQLCA中取出这些状态信息，据此决定接下来执行的语句**



SQL通信区

- SQLCA使用方法：
 - 定义SQLCA
 - 用EXEC SQL INCLUDE SQLCA定义
 - 使用SQLCA
 - SQLCA中有一个存放每次执行SQL语句后返回代码的变量SQLCODE
 - 如果SQLCODE等于预定义的常量SUCCESS，则表示SQL语句成功，否则表示出错
 - 应用程序每执行完一条SQL 语句之后都应该测试一下SQLCODE的值，以了解该SQL语句执行情况并做相应处理



主变量

- 嵌入式SQL语句中可以使用主语言的程序变量来输入或输出数据
- 在SQL语句中使用的主语言程序变量简称为主变量 (Host Variable)

(1) 主变量的类型

- 输入主变量
- 输出主变量
- 一个主变量有可能既是输入主变量又是输出主变量



(2) 指示变量

- 一个主变量可以附带一个指示变量 (Indicator Variable)
- 实际上也是一类SQL变量, 它被用来管理与其相关联的宿主变量 (即在SQL语句中充当输入或输出的变量)。每一个宿主变量都可定义一个指示变量, 主要用于处理空值 (NULL), 指示器变量的说明基本同一般SQL变量一样, 但必须定义成2字节的整型, 如SHORTINT。在SQL语句中引用时, 其前也应加 " : " (冒号), 而且必须附在其相关联的宿主变量之后, 在C语句中, 可独立使用。当指示变量为-1时, 表示空值



- 在SQL语句中使用主变量和指示变量的方法

- 1) 说明主变量和指示变量

BEGIN DECLARE SECTION

.....

..... (说明主变量和指示变量)

END DECLARE SECTION

- 2) 使用主变量

- 说明之后的主变量可以在SQL语句中任何一个能够使用表达式的地方出现

- 为与数据库对象名区别，SQL语句中的主变量名前加冒号（:）作为标志

- 3) 使用指示变量

- 指示变量前也必须加冒号标志

- 必须紧跟在所指主变量之后



- 在SQL语句之外(主语言语句中)使用主变量和指示变量的方法

– 可以直接引用，不必加冒号

```
EXEC SQL BEGIN DECLARE SECTION
```

```
Int      dept_number;
```

```
Short    ind_num;
```

```
Char     emp_name;
```

```
EXEC SQL END DECLARE SECTION
```

```
Scanf("90d %s", &dept_number, dept_name);
```

```
If (dept_number == 0)
```

```
    ind_num = -1;
```

```
Else
```

```
    ind_num = 0;
```

```
EXEC SQL INSERT INTO DEPT (DEPTNO, DNAME) VALUES (:dept_number:ind_num, :dept_name);
```

其中Ind_num是
dept_number的指示变量。
当输入的dept_number 值是
0时，则向DEPT 表的
DEPTNO列插入空值



- 为什么要使用游标
 - SQL语言与主语言具有不同数据处理方式
 - SQL语言是面向集合的，一条SQL语句原则上可以产生或处理多条记录
 - 主语言是面向记录的，一组主变量一次只能存放一条记录
 - 仅使用主变量并不能完全满足SQL语句向应用程序输出数据的要求
 - 嵌入式SQL引入了游标的概念，用来协调这两种不同的处理方式
- SQL的集合处理方式与宿主语言单记录处理方式之间的协调

SQL：一次一集合。

C语言：一次一记录。

游标：在查询结果的记录集合中移动的指针。

若一个SQL语句返回单个元组，则不用游标。

若一个SQL语句返回多个元组，则使用游标。



- **游标**

- **游标是系统为用户开设的一个数据缓冲区，存放SQL语句的执行结果**
- **每个游标区都有一个名字**
- **用户可以用SQL语句逐一从游标中获取记录，并赋给主变量，交由主语言进一步处理**



SQL语言与主语言之间的通信

- 建立数据库连接

EXEC SQL CONNECT TO *target* [AS *connection-name*] [USER *user-name*];

*target*是要连接的数据库服务器：

- 常见的服务器标识串，如<dbname>@<hostname>:<port>
- 包含服务器标识的SQL串常量
- DEFAULT

*connection-name*是指定的连接名，连接必须是一个有效的标识符
在整个程序内只有一个连接时可以不指定连接名

- 关闭数据库连接

EXEC SQL DISCONNECT [*connection*];

- 程序运行过程中可以修改当前连接：

EXEC SQL SET CONNECTION *connection-name* | DEFAULT;



【例】 依次检查某个系的学生记录，交互式更新某些学生年龄。

```
EXEC SQL BEGIN DECLARE SECTION; /*主变量说明开始*/
```

```
    char deptname[64];
```

```
    char HSno[64];
```

```
    char HSname[64];
```

```
    char HSsex[64];
```

```
    int    HSage;
```

```
    int    NEWAGE;
```

```
EXEC SQL END DECLARE SECTION;    /*主变量说明结束*/
```

```
long SQLCODE;
```

```
EXEC SQL INCLUDE sqlca;          /*定义SQL通信区*/
```



```
int main(void)                                /*C语言主程序开始*/
{
    int    count = 0;
    char  yn;                                  /*变量yn代表yes或no*/
    printf("Please choose the department name(CS/MA/IS): ");
    scanf("%s", deptname);                    /*为主变量deptname赋值*/
    EXEC SQL CONNECT TO TEST@localhost:54321 USER
    "SYSTEM" /"MANAGER";                     /*连接数据库TEST*/
    EXEC SQL DECLARE SX CURSOR FOR            /*定义游标*/
        SELECT Sno, Sname, Ssex, Sage        /*SX对应语句的执行结果*/
        FROM Student
        WHERE SDept = :deptname;
    EXEC SQL OPEN SX;                         /*打开游标SX便指向查询结果的第一行*/
```



```
for ( ;; )                /*用循环结构逐条处理结果集中的记录*/
{
    EXEC SQL FETCH SX INTO :HSno, :HSname, :HSsex,:HSage;
                        /*推进游标，将当前数据放入主变量*/
    if (sqlca.sqlcode != 0) /* sqlcode != 0,表示操作不成功*/
        break;          /*利用SQLCA中的状态信息决定何时退出循环*/
    if(count++ == 0)      /*如果是第一行的话，先打出行头*/
        printf("\n%-10s %-20s %-10s %-10s\n", "Sno", "Sname", "Ssex", "Sage");
        printf("%-10s %-20s %-10s %-10d\n", HSno, HSname, HSsex, HSage);
                        /*打印查询结果*/
        printf("UPDATE AGE(y/n)?"); /*询问用户是否要更新该学生的年龄*/
    do{
        scanf("%c",&yn);
    }
    while(yn != 'N' && yn != 'n' && yn != 'Y' && yn != 'y');
```



```
if (yn == 'y' || yn == 'Y')                /*如果选择更新操作*/
{
    printf("INPUT NEW AGE:");
    scanf("%d",&NEWAGE);                /*用户输入新年龄到主变量中*/
    EXEC SQL UPDATE Student                /*嵌入式SQL*/
        SET Sage = :NEWAGE
        WHERE CURRENT OF SX ;
    }                                     /*对当前游标指向的学生年龄进行更新*/
}

EXEC SQL CLOSE SX;                        /*关闭游标SX不再和查询结果对应*/
EXEC SQL COMMIT WORK;                     /*提交更新*/
EXEC SQL DISCONNECT TEST;                 /*断开数据库连接*/
}
```




8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不使用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.6 小结



不使用游标的语句

- 说明性语句
 - 数据定义语句
 - 数据控制语句
 - 查询结果为单记录的SELECT语句
 - 非CURRENT形式的UPDATE语句
 - 非CURRENT形式的DELETE语句
 - INSERT语句
- 最简单的一类语句，不需返回结果，不使用主变量，在主语言中只需加前缀EXEC SQL和语句结束符即可。
- 一般均使用主变量



不使用游标的语句

查询结果为单记录的SELECT语句

例 根据主变量的值查找学生的信息

```
EXEC SQL SELECT S# , C# , G
```

```
INTO :SNO , :CNO , :G :GID
```

```
FROM SC
```

```
WHERE S# = :GS AND C# = :GC ;
```

查询结果存入这三个主变量中

指示变量，
<0说明取得的G为空值

- SELECT语句的INTO、WHERE、HAVING子句中可使用主变量
- 可在INTO子句中使用指示变量，以指明某字段是否空值
- 若SQLCODE=100，说明没有满足条件的记录
- 当查询到的记录多于1条时，在SQLCODE中返回错误信息



不使用游标的语句

非CURRENT形式的UPDATE语句

例 将计算机系全体学生的成绩置为空值

GID = -1 ;

EXEC SQL UPDATE SC

SET G = :GG :GID

WHERE "CS" =

(SELECT SD

FROM S

WHERE S.S#=SC.S#);

这里使用了值为负的指示变量GID,
主变量GG可为任意值

- WHERE、SET子句中可以使用主变量，同时SET子句中还可以使用指示变量
- 通过检查SQLCA的值，判别更新是否成功



不使用游标的语句

非CURRENT形式的DELETE语句

例 学号在主变量X1和X2之间的学生已毕业，删除他们的信息。

先删除他们的选课信息

```
EXEC SQL DELETE
```

```
FROM SC
```

```
WHERE S# BETWEEN :X1 AND :X2;
```

再删除他们的基本情况信息

```
EXEC SQL DELETE
```

```
FROM S
```

```
WHERE S# BETWEEN :X1 AND :X2;
```

WHERE子句中可使用主变量



不使用游标的语句

INSERT语句

例 插入一条学生选课记录

GID = -1 ;

EXEC SQL INSERT

INTO SC (S# , C# , G)

VALUES (:SNO, :CNO, :GG :GID) ;

欲插入记录的值由主语言存放在三个主变量中，GID指示成绩字段值为空值，GG可为任意值



8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不使用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.6 小结



使用游标的语句

下列情况必须使用游标

- 查询结果为多条记录的SELECT 语句
- CURRENT形式的UPDATE语句
- CURRENT形式的DELETE语句



需要游标的数据库操作

- **当select语句的结果中包含多个元组时，使用游标可以逐个存取这些元组**

活动集：select语句返回的元组的集合。

当前行：活动集中当前处理的那一行。游标即是指向当前行的指针。

游标分类：

滚动游标：游标的位置可以来回移动，可在活动集中取任意元组。

非滚动游标：只能在活动集中顺序地取下一个元组。

更新游标：数据库对游标指向的当前行加锁，当程序读下一行数据时，本行数据解锁，下一行数据加锁。



■ 游标的使用方法：

1、定义游标：定义一个游标，使之对应一个select语句。

```
EXEC_SQL  DECLARE <游标名> CURSOR FOR  
<SELECT语句>      END_EXEC
```

- 是一条说明性语句，这时DBMS并不执行SELECT指定的查询操作。

2、打开游标：打开一个游标，执行游标对应的查询，结果集合为该游标的活动集。

```
EXEC SQL OPEN <游标名>  //执行查询  
  
END_EXEC
```



3、游标推进语句：在活动集中将游标移到特定的行，并取出该行数据放到相应的宿主变量中

**EXEC SQL FETCH [NEXT|PRIOR|FIRST|LAST] FROM <游标名
> INTO <共享变量名> END_EXEC**

打开游标时，游标中当前行的位置逻辑上应位于第一行。这使不同的提取选项具有下列行为，如果这是打开游标后的第一次提取操作：

FETCH FIRST：提取游标中的第一行。

FETCH NEXT：提取游标中的下一行。

FETCH PRIOR：提取游标中的前一行。



4、游标关闭语句：关闭游标，释放活动集及其所占资源，使它不再和查询结果相联系。需要再使用该游标时，执行open语句。

EXEC SQL CLOSE <游标名> END_EXEC

5. 释放游标

游标结构本身也会站用一定的计算机资源，所以在使用完游标后，应该将游标释放，以回收资源。

语句:

DEALLOCATE cursor_name



CURRENT形式的UPDATE语句和DELETE语句的用途

非CURRENT形式的UPDATE语句和DELETE语句

- 面向集合的操作
- 一次修改或删除所有满足条件的记录

如果只想修改或删除其中某个记录

- 用带游标的SELECT语句查出所有满足条件的记录从中进一步找出要修改或删除的记录
- 用CURRENT形式的UPDATE语句和DELETE语句修改或删除
- UPDATE语句和DELETE语句中要用子句

WHERE CURRENT OF <游标名>

表示修改或删除的是最近一次取出的记录，即游标指针指向的记录



不能使用CURRENT形式的UPDATE语句DELETE语句

- 当游标定义中的SELECT语句带有UNION或ORDER BY子句
- 该SELECT语句相当于定义了一个不可更新的视图



8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不使用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.6 小结



- **静态嵌入式SQL**

- 静态嵌入式SQL语句能够满足一般要求
- 无法满足要到执行时才能够确定要提交的SQL语句

- **动态嵌入式SQL**

- 允许在程序运行过程中临时“**组装**”SQL语句
- 支持动态组装SQL语句和动态参数两种形式



- **一、使用SQL语句主变量**
- **二、动态参数**
- **三、执行准备好的语句（EXECUTE）**



- **SQL语句主变量：**
 - **程序主变量包含的内容是SQL语句的内容（动态），而不是原来保存数据的输入或输出变量（静态）**
 - **SQL语句主变量在程序执行期间可以设定不同的SQL语句，然后立即执行**



【例】创建基本表TEST

```
EXEC SQL BEGIN DECLARE SECTION;
```

```
const char *stmt = "CREATE TABLE test(a int);";
```

```
/* SQL语句主变量 */
```

```
EXEC SQL END DECLARE SECTION;
```

```
... ..
```

```
EXEC SQL EXECUTE IMMEDIATE :stmt;
```

```
/* 执行语句 */
```



- **动态参数**
 - SQL语句中的可变元素
 - 使用参数符号(?)表示该位置的数据在运行时设定
- **和主变量的区别**
 - 动态参数的输入不是编译时完成绑定
 - 而是通过 (prepare)语句准备主变量和执行语句(execute)时绑定数据或主变量来完成



- 使用动态参数的步骤：

- 1.声明SQL语句主变量。

SQL语句的主变量的值包含动态参数（ ? ）

- 2.准备SQL语句(PREPARE)。

EXEC SQL PREPARE <语句名> FROM <SQL语句主变量>;

- 3.执行准备好的语句(EXECUTE)

EXEC SQL EXECUTE <语句名> [INTO <主变量表>] [USING <主变量或常量>];



【例】向TEST中插入元组。

```
EXEC SQL BEGIN DECLARE SECTION;  
const char *stmt = "INSERT INTO test VALUES(?);";  
        /*声明SQL主变量 */  
EXEC SQL END DECLARE SECTION;  
  
... ..  
EXEC SQL PREPARE mystmt FROM :stmt; /* 准备语句 */  
  
... ..  
EXEC SQL EXECUTE mystmt USING 100; /* 执行语句 */  
EXEC SQL EXECUTE mystmt USING 200; /* 执行语句 */
```



8.1.1 嵌入式SQL的处理过程

8.1.2 嵌入式SQL语句与主语言之间的通信

8.1.3 不使用游标的SQL语句

8.1.4 使用游标的SQL语句

8.1.5 动态SQL

8.1.6 小结



- **在嵌入式SQL中，SQL语句与主语言语句分工非常明确**
 - **SQL语句**
 - **直接与数据库打交道，取出数据库中的数据。**
 - **主语言语句**
 - **控制程序流程**
 - **对取出的数据做进一步加工处理**



- **SQL语言是面向集合的，一条SQL语句原则上可以产生或处理多条记录**
- **主语言是面向记录的，一组主变量一次只能存放一条记录**
 - **仅使用主变量并不能完全满足SQL语句向应用程序输出数据的要求**
 - **嵌入式SQL引入了游标的概念，用来协调这两种不同的处理方式**



8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

8.5 OLE DB

8.6 JDBC编程

8.7 小结



8.2.1 PL/SQL的块结构

8.2.2 变量常量的定义

8.2.3 流程控制



- **PL/SQL :**
 - **SQL的扩展**
 - **增加了过程化语句功能**
 - **基本结构是块**
 - **块之间可以互相嵌套**
 - **每个块完成一个逻辑操作**



- **PL/SQL块的基本结构：**

1.定义部分

DECLARE ：定义的变量、常量等只能在该基本块中使用

变量、常量、游标、异常等：当基本块执行结束时，定义就不再存在



- **PL/SQL块的基本结构**

2.执行部分

BEGIN

-----SQL语句、PL/SQL的流程控制语句

EXCEPTION

-----异常处理部分

END ;



8.2.1 PL/SQL的块结构

8.2.2 变量常量的定义

8.2.3 流程控制



1. PL/SQL中定义变量的语法形式是:

变量名 数据类型 [[NOT NULL] := 初值表达式] 或

变量名 数据类型 [[NOT NULL] 初值表达式]

2. 常量的定义类似于变量的定义:

常量名 数据类型 CONSTANT := 常量表达式

常量必须要给一个值，并且该值在存在期间或常量的作用域内不能改变。如果试图修改它，PL/SQL将返回一个异常。

3. 赋值语句

变量名称:=表达式



8.2.1 PL/SQL的块结构

8.2.2 变量常量的定义

8.2.3 流程控制



- **PL/SQL 功能：**

- 一、条件控制语句**

- 二、循环控制语句**

- 三、错误处理**



一、条件控制语句

IF-THEN , IF-THEN-ELSE和嵌套的IF语句

1. IF *condition* THEN

Sequence_of_statements,

END IF

2. IF condition THEN

Sequence_of_statements1;

ELSE

Sequence_of_statements2;

END IF;

3. 在THEN和ELSE子句中还可以再包括IF语句，即IF语句可以嵌套



二、循环控制语句

LOOP , WHILE-LOOP和FOR-LOOP

1.最简单的循环语句LOOP

LOOP

Sequence_of_statements;

END LOOP;

多数数据库服务器的PL/SQL都提供EXIT、BREAK或LEAVE等循环结束语句，
保证LOOP语句块能够结束。



二、循环控制语句

2. WHILE-LOOP

WHILE condition LOOP

Sequence_of_statements;

END LOOP;

- 每次执行循环体语句之前，首先对条件进行求值
- 如果条件为真，则执行循环体内的语句序列。
- 如果条件为假，则跳过循环并把控制传递给下一个语句



二、循环控制语句

3. FOR-LOOP

```
FOR count IN [ REVERSE ] bound1 ... bound2 LOOP  
    Sequence_of_statements;  
END LOOP;
```

执行过程：循环变量设为下界bound1，检查循环变量是否小于上界bound2，如果是执行循环体，循环变量加1，如果否，退出循环。当设置REVERSE，表示bound1为上界，bound2为下界，每次执行循环体后循环变量减1。



三、错误处理：

- 如果PL/SQL在执行时出现异常，则应该让程序在产生异常的语句处停下来，根据异常的类型去执行异常处理语句
- SQL标准对数据库服务器提供什么样的异常处理做出了建议，要求过程化SQL管理器 PL/SQL管理器提供完善的异常处理机制



8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

8.5 OLE DB

8.6 JDBC编程

8.7 小结



8.3.1 存储过程

8.3.2 函数

8.3.3 过程化SQL中的游标



过程化SQL块类型

- **匿名块** 每次执行时都要进行编译，它不能被存储到数据库中，也不能在其他过程化SQL块中调用
- **命名块** 编译后保存在数据库中，可以被反复调用，运行速度较快，过程和函数是命名块



存储过程：由过程化SQL语句书写的过程，经编译和优化后存储在数据库服务器中，使用时只要调用即可。

存储过程的优点

- **（1）运行效率高**
- **（2）降低了客户机和服务器之间的通信量**
- **（3）方便实施企业规则**



存储过程的用户接口

- (1) 创建存储过程**
- (2) 执行存储过程**
- (3) 修改存储过程**
- (4) 删除存储过程**



(1) 创建存储过程

CREATE OR REPLACE PROCEDURE 过程名([参数1, 参数2,...])

AS <过程化SQL块> ;

- **过程名**：数据库服务器合法的对象标识
- **参数列表**：用名字来标识调用时给出的参数值，必须指定值的数据类型。参数也可以定义输入参数、输出参数或输入/输出参数，默认为输入参数
- **过程体**：是一个<过程化SQL块>，包括声明部分和可执行语句部分



【例】利用存储过程来实现下面的应用：从账户1转指定数额的款项到账户2中。

```
CREATE OR REPLACE PROCEDURE TRANSFER(inAccount  
INT,outAccount INT,amount FLOAT)
```

```
/*定义存储过程TRANSFER，其参数为转入账户、转出账户、转账额度*/
```

```
AS DECLARE /*定义变量*/
```

```
totalDepositOut Float;
```

```
totalDepositIn Float;
```

```
inAccountnum INT;
```



BEGIN

/*检查转出账户的余额 */

**SELECT Total INTO totalDepositOut FROM Accout
WHERE accountnum=outAccount;**

IF totalDepositOut IS NULL THEN

/*如果转出账户不存在或账户中没有存款*/

ROLLBACK; /*回滚事务*/

RETURN;

END IF;



UPDATE Account SET total=total-amount

WHERE accountnum=outAccount;

/* 修改转出账户余额，减去转出额 */

UPDATE Account SET total=total + amount

WHERE accountnum=inAccount;

/* 修改转入账户余额，增加转入额 */

COMMIT;

/* 提交转账事务 */

END;



(2) 执行存储过程

CALL/PERFORM PROCEDURE 过程名([参数1,参数2,...]) ;

使用CALL或者PERFORM等方式激活存储过程的执行

在过程化SQL中，数据库服务器支持在过程体中调用其他存储过程

【例】从账户01003815868转10000元到01003813828账户中。

CALL PROCEDURE

TRANSFER(01003813828,01003815868,10000);



(3) 修改存储过程

ALTER PROCEDURE 过程名1 RENAME TO 过程名2;

(4) 删除存储过程

DROP PROCEDURE 过程名();



8.3.1 存储过程

8.3.2 函数

8.3.3 过程化SQL中的游标



- **函数和存储过程的异同**
 - **同：都是持久性存储模块**
 - **异：函数必须指定返回的类型**



1. 函数的定义语句格式

CREATE OR REPLACE FUNCTION 函数名 ([参数1,参数2,...]) RETURNS <类型> AS <过程化SQL块>;

2. 函数的执行语句格式

CALL/SELECT 函数名 ([参数1,参数2,...]);

3. 修改函数

—重命名

ALTER FUNCTION 过程名1 RENAME TO 过程名2;

—重新编译

ALTER FUNCTION 过程名 COMPILE;



8.3.1 存储过程

8.3.2 函数

8.3.3 过程化SQL中的游标



在过程化SQL中如果查询返回多条记录时，就要使用游标对结果集进行处理。在存储过程中可以定义普通游标、REFCURSOR类型游标、带参数游标等。

【例】定义存储过程，多次打开游标并获取游标当前记录

```
CREATE OR REPLACE PROCEDURE pro_cursor() AS DECLARE  
  
    cno CHAR(3);  
  
    cname CHAR(8);
```




```
CURSOR mycursor(leaderno CHAR(3)) FOR /*说明带参数游标mycursor*/
```

```
SELECT Ino, Iname FROM leader WHERE Ino=leaderno;
```

```
BEGIN
```

```
/*mycursor能检索leader表中具有参数leaderno的记录*/
```

```
OPEN mycursor( 'L01' ) /*使用参数 'L01' 打开游标*/
```

```
FETCH mycursor INTO cno, cname; /*获取Ino= 'L01' 的游标元组*/
```

```
INSERT INTO temp(Ino, Iname) VALUES(cno, cname)
```

```
/*将游标元组插入临时表中*/
```



CLOSE mycursor; /*关闭游标*/

OPEN mycursor('L02'); /*使用新的参数L02重新打开游标*/

FETCH mycursor INTO cno, cname;

INSERT INTO temp(lno, lname) VALUES(cno, cname)

CLOSE mycursor;

END



8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

8.5 OLE DB

8.6 JDBC编程

8.7 小结



8.4.1 ODBC概述

8.4.2 ODBC工作原理概述

8.4.3 ODBC API 基础

8.4.4 ODBC的工作流程



- ODBC产生的原因：
 - 由于**不同的数据库管理系统**的存在，在某个RDBMS下编写的应用程序就不能在另一个RDBMS下运行
 - 许多应用程序需要共享多个部门的数据资源，访问不同的RDBMS（同一公司的不同区域的分公司）



- **ODBC :**
 - 是微软公司开放服务体系(Windows Open Services Architecture , WOSA)中有关数据库的一个组成部分
 - 提供了一组访问数据库的标准API
- **ODBC约束力（主要作用）：**
 - 规范应用开发
 - 规范RDBMS应用接口



8.4.1 ODBC概述

8.4.2 ODBC工作原理概述

8.4.3 ODBC API 基础

8.4.4 ODBC的工作流程



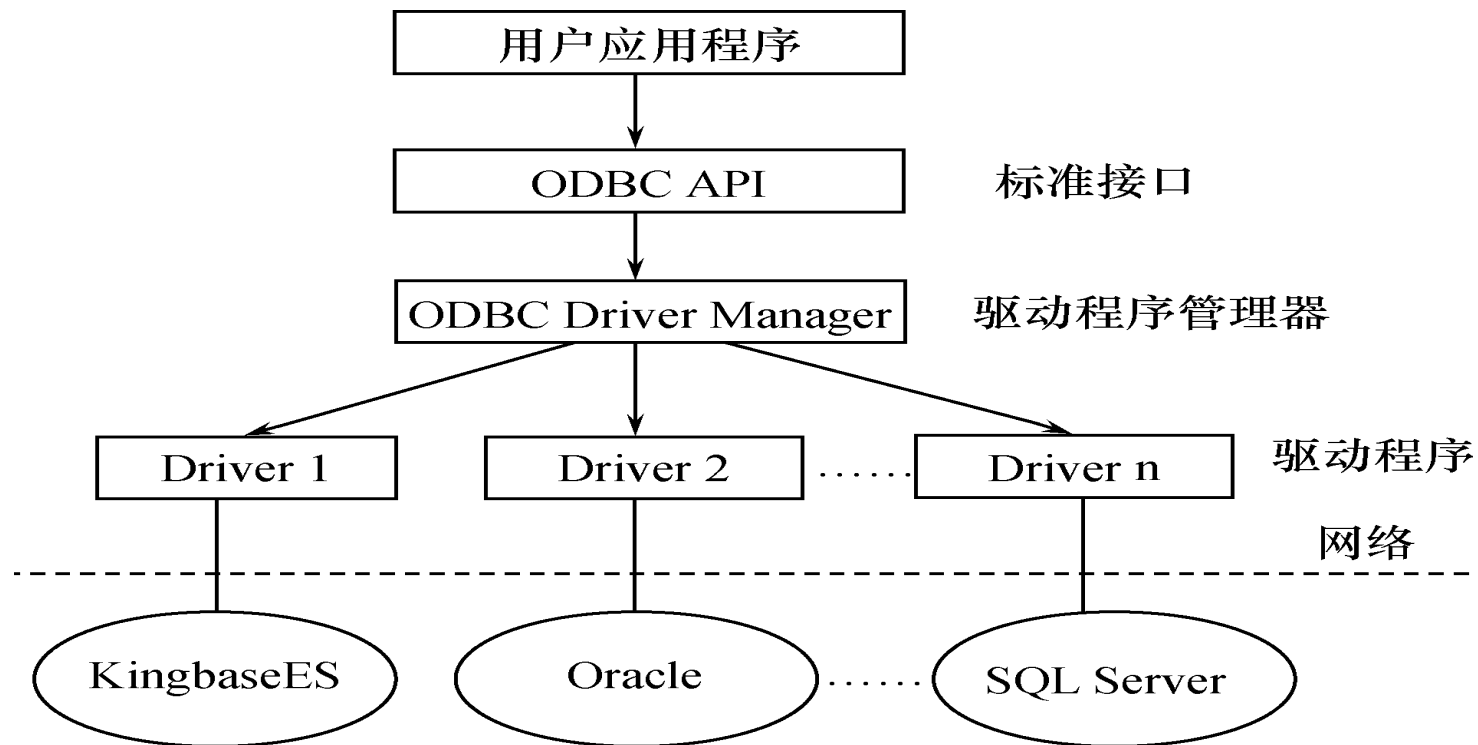
- ODBC应用系统的体系结构：

一、 用户应用程序

二、 驱动程序管理器

三、 数据库驱动程序

四、 ODBC数据源管理





1、用户应用程序

- **ODBC应用程序包括的内容：**
 - (1) 请求连接数据库 ；**
 - (2) 向数据源发送SQL语句 ；**
 - (3) 为SQL语句执行结果分配存储空间 ， 定义所读取的数据格式 ；**
 - (4) 获取数据库操作结果 ， 或处理错误 ；**
 - (5) 进行数据处理并向用户提交处理结果 ；**
 - (6) 请求事务的提交和回滚操作 ；**
 - (7) 断开与数据源的连接。**



2、ODBC驱动程序管理器

- 驱动程序管理器：用来管理各种驱动程序
 - 包含在ODBC32.DLL中，对用户**透明**
 - 管理应用程序和驱动程序之间的通信
 - 建立、配置或删除数据源并查看系统当前所安装的数据库ODBC驱动程序
 - 主要功能：
 - 装载ODBC驱动程序
 - 选择和连接正确的驱动程序
 - 管理数据源
 - 检查ODBC调用参数的合法性
 - 记录ODBC函数的调用等



3、数据库驱动程序

- ODBC通过**驱动程序**来提供应用系统与数据库平台的独立性
- ODBC应用程序**不能**直接存取数据库
 - 其各种操作请求由驱动程序管理器**提交给某个RDBMS**的ODBC驱动程序
 - 通过调用驱动程序所支持的函数来存取数据库。
 - 数据库的操作结果也通过驱动程序返回给应用程序。
 - 如果应用程序要操纵不同的数据库，就要动态地链接到不同的驱动程序上。



3、数据库驱动程序

- ODBC驱动程序类型：

- 单束

- 数据源和应用程序在同一台机器上
 - 驱动程序直接完成对数据文件的I/O操作
 - 驱动程序相当于数据管理器

- 多束

- 支持客户机/服务器、客户机/应用服务器/数据库服务器等网络环境下的数据访问
 - 由驱动程序完成数据库访问请求的提交和结果集接收
 - 应用程序使用驱动程序提供的结果集管理接口操纵执行后的结果数据



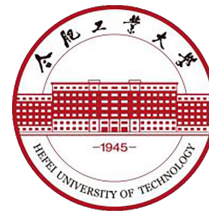
4、ODBC数据源管理

- 数据源：是最终用户需要访问的数据，包含了数据库位置和数据库类型等信息，是一种数据连接的抽象
- 数据源对最终用户是透明的
 - ODBC给**每个被访问的数据源指定唯一的数据源名**（Data Source Name，简称DSN），并映射到所有必要的、用来存取数据的低层软件
 - 在连接中，用数据源名来代表用户名、服务器名、所连接的数据库名等
 - 最终用户无需知道DBMS或其他数据管理软件、网络以及有关ODBC驱动程序细节



【例】 假设某个学校在MS SQL Server和KingbaseES上创建了两个数据库：学校人事数据库和教学科研数据库。

- 学校的信息系统要从这两个数据库中存取数据**
- 为方便与两个数据库连接，为学校人事数据库创建一个数据源名PERSON，为教学科研数据库创建一个名为EDU的数据源。**
- 当要访问每一个数据库时，只要与PERSON和EDU连接即可,不需要记住使用的驱动程序、服务器名称、数据库名（屏蔽底层数据库的差异）**



8.4.1 ODBC概述

8.4.2 ODBC工作原理概述

8.4.3 ODBC API 基础

8.4.4 ODBC的工作流程



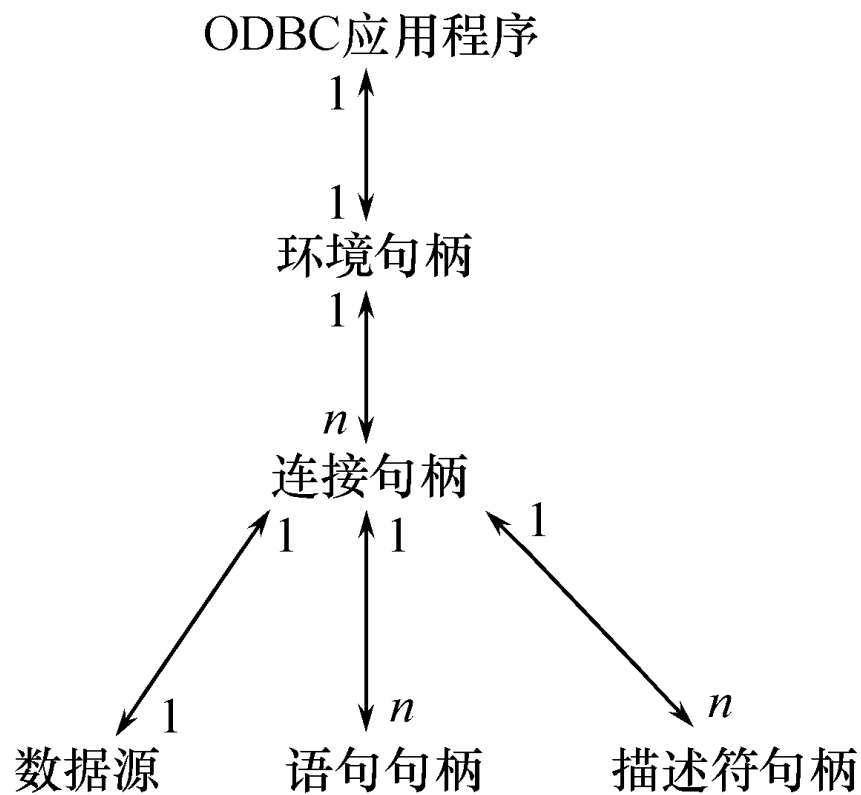
- **ODBC 应用程序接口的一致性**
 - **API一致性**
 - **API一致性级别有核心级、扩展1级、扩展2级**
 - **语法一致性**
 - **语法一致性级别有最低限度SQL语法级、核心SQL语法级、扩展SQL语法级**



- **ODBC 3.0 标准提供了76个函数接口：**
 - 分配和释放环境句柄、连接句柄、语句句柄；
 - 连接函数（SQLDriverconnect等）；
 - 与信息相关的函数（如获取描述信息函数SQLGetinfo、SQLGetFuction）；
 - 事务处理函数（如SQLEndTran）；
 - 执行相关函数（SQLExecdirect、SQLExecute等）；
 - 编目函数，ODBC 3.0提供了11个编目函数如SQLTables、SQLColumn等，应用程序可以通过对编目函数的调用来获取数据字典的信息如权限、表结构等



- 句柄是32位整数值，代表一个指针
- ODBC 3.0中句柄分类：
 - 环境句柄
 - 连接句柄
 - 语句句柄
 - 描述符句柄



应用程序句柄之间的关系



【说明】

1. 每个ODBC应用程序需要建立一个ODBC环境，分配一个环境句柄，存取数据的全局性背景如环境状态、当前环境状态诊断、当前在环境上分配的连接句柄等；
2. 一个环境句柄可以建立多个连接句柄，每一个连接句柄实现与一个数据源之间的连接；
3. 在一个连接中可以建立多个语句子柄，它不只是一个SQL语句，还包括SQL语句产生的结果集以及相关的信息等；
4. 在ODBC 3.0中又提出了描述符句柄的概念，它是描述SQL语句的参数、结果集列的元数据集合。



ODBC数据类型：

–SQL数据类型：用于数据源

–C数据类型：用于应用程序的C代码

应用程序可以通过SQLGetTypeInfo来获取不同的驱动程序对于数据类型的支持情况

SQL数据类型和C数据类型之间的转换规则

	SQL数据类型	C数据类型
SQL数据类型	数据源之间转换	应用程序变量传送到语句参数（SQLBindparameter）
C数据类型	从结果集列中返回到应用程序变量（SQLBindcol）	应用程序变量之间转换

输入变量

输出变量



8.4.1 ODBC概述

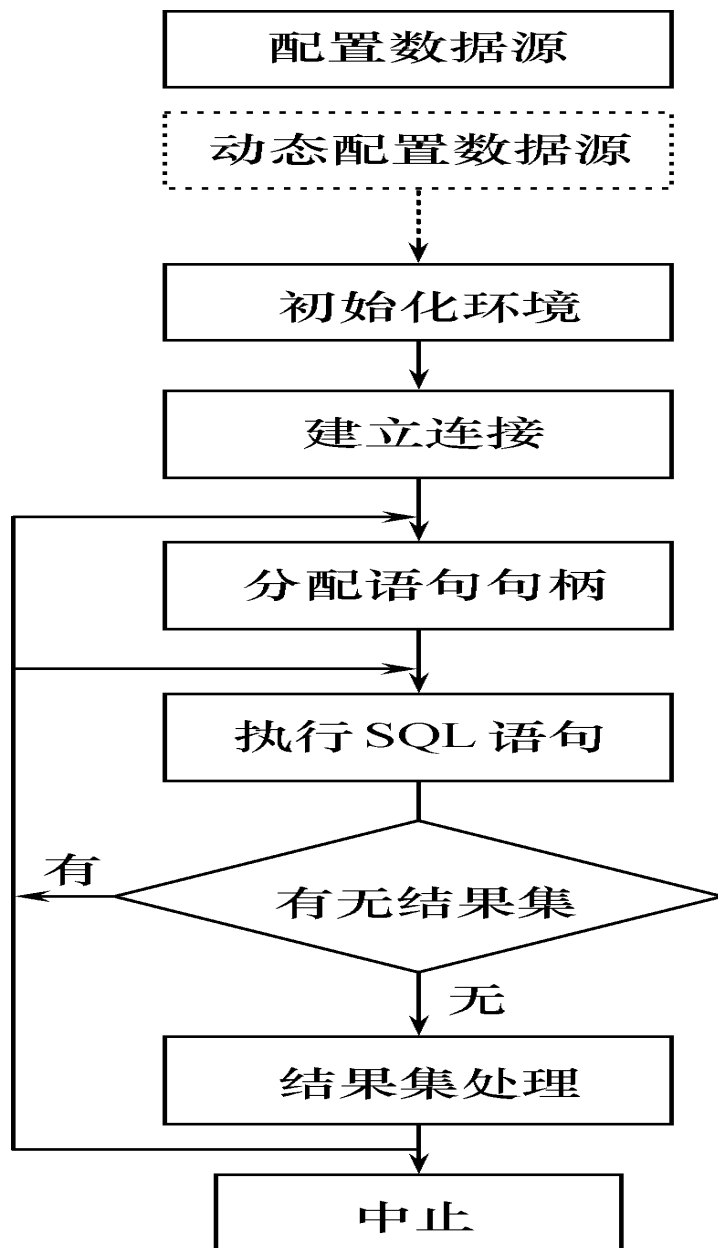
8.4.2 ODBC工作原理概述

8.4.3 ODBC API 基础

8.4.4 ODBC的工作流程



ODBC的工作流程：





【例】将KingbaseES数据库中Student表的数据备份到SQL SERVER数据库中。

- **该应用涉及两个不同的RDBMS中的数据源**
- **使用ODBC来开发应用程序，只要改变应用程序中连接函数（SQLConnect）的参数，就可以连接不同RDBMS的驱动程序，连接两个数据源**



- 在应用程序运行前，已经在KingbaseES和SQL SERVER中分别建立了STUDENT关系表

CREATE TABLE Student

(Sno CHAR (9) PRIMARY KEY ,

Sname CHAR (20) UNIQUE

Ssex CHAR (2) ,

Sage SMALLINT ,

Sdept CHAR (20)

) ;



- 应用程序要执行的操作是：

- 在KingbaseES上执行SELECT * FROM STUDENT ;

- 把获取的结果集，通过多次执行

**INSERT INTO STUDENT (Sno , Sname , Ssex , Sage ,
Sddept) VALUES (? , ? , ? , ? , ?) ;**

- 插入到SQL SERVER的STUDENT表中



- **操作步骤：**
 - 一、配置数据源**
 - 二、初始化环境**
 - 三、建立连接**
 - 四、分配语句句柄**
 - 五、执行SQL语句**
 - 六、结果集处理**
 - 七、中止处理**



1、配置数据源

- 配置数据源两种方法：

- (1)运行数据源管理工具来进行配置；

- (2)使用Driver Manager 提供的ConfigDsn函数来增加、修改或删除数据源

- 在【例】中，采用了第一种方法创建数据源。因为要同时用到KingbaseES和SQL Server，所以分别建立两个数据源，将其取名为KingbaseES ODBC和SQLServer。



2、初始化环境

- 没有和具体的驱动程序相关联，由Driver Manager来进行控制，并配置环境属性
- 应用程序通过调用连接函数和某个数据源进行连接后，Driver Manager才调用所连的驱动程序中的SQLAllocHandle，来真正分配环境句柄的数据结构



3、建立连接

- 应用程序调用SQLAllocHandle分配连接句柄，通过SQLConnect、SQLDriverConnect或SQLBrowseConnect与数据源连接
- SQLConnect连接函数，输入参数为：
 - 配置好的数据源名称
 - 用户ID
 - 口令



4、分配语句句柄

- 处理任何SQL语句之前，应用程序还需要首先分配一个语句句柄
- 语句句柄含有具体的SQL语句以及输出的结果集等信息
- 应用程序还可以通过SQLtStmtAttr来设置语句属性（也可以使用默认值）
- 结果集绑定的方式为按列绑定



5、执行SQL语句

- 应用程序处理SQL语句的两种方式：
 - 预处理（SQLPrepare、SQLExecute适用于语句的多次执行）
 - 直接执行（SQLExecdirect）
- 如果SQL语句含有参数，应用程序为每个参数调用SQLBindParameter，并把它们绑定至应用程序变量
- 应用程序可以直接通过改变应用程序缓冲区的内容从而在程序中动态的改变SQL语句的具体执行



5、执行SQL语句

- 应用程序根据语句的类型进行的处理
 - 有结果集的语句（select或是编目函数），则进行结果集处理。
 - 没有结果集的函数，可以直接利用本语句句柄继续执行新的语句或是获取行计数（本次执行所影响的行数）之后继续执行。
- 使用SQLExecdirect获取KingbaseES中的结果集，并将结果集根据各列不同的数据类型绑定到用户程序缓冲区
- 在插入数据时，采用了预编译的方式，首先通过SQLPrepare来预处理SQL语句，将每一列绑定到用户缓冲区
- 应用程序可以直接修改结果集缓冲区的内容



6、结果集处理

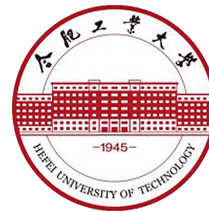
- 应用程序可以通过SQLNumResultCols来获取结果集中的列数
- 通过SQLDescribeCol或是SQLColAttribute函数来获取结果集每一列的名称、数据类型、精度和范围



6、结果集处理

- 结果集处理步骤：

- ODBC游标打开方式不同于嵌入式SQL，不是显式声明而是系统自动产生一个游标（Cursor），当结果集刚刚生成时，游标指向第一行数据之前
- 应用程序通过SQLBindCol，把查询结果绑定到应用程序缓冲区中，通过SQLFetch或是SQLFetchScroll来移动游标获取结果集中的每一行数据
- 对于如图像这类特别的数据类型当一个缓冲区不足以容纳所有的数据时，可以通过SQLGetData分多次获取
- 最后通过SQLClosecursor来关闭游标



7、中止处理

- **应用程序中止步骤：**
 - **首先释放语句句柄**
 - **释放数据库连接**
 - **与数据库服务器断开**
 - **释放ODBC环境**



小结

- **ODBC目的：为了提高应用系统与数据库平台的独立性，使得应用系统的移植变得容易**
- **ODBC优点：**
 - **使得应用系统的开发与数据库平台的选择、数据库设计等工作并行进行**
 - **方便移植**
 - **大大缩短整个系统的开发时间**



8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

8.5 OLE DB

8.6 JDBC编程

8.7 小结



- **什么是OLE DB**

- **OLE DB对象链接与嵌入式数据库，是微软提出的数据库访问标准。是基于组件对象模型（COM）来访问各种数据源的ActiveX的通用接口**
- **OLE DB可以连接数据库，也可以是文本文件、Excel表格等各种不同格式的数据存储（ODBC只能连接关系数据库）**



- **OLE DB的结构**

OLE DB包括了消费者和提供者

- **消费者**：利用OLE DB提供的接口访问数据库数据的客户端应用程序或其他工具（**要访问数据**）
- **提供者**：提供者是一个由COM组件构成的数据访问中介，位于数据库和消费者之间。（**要提供数据**）
 - （1）**服务提供者**。这类提供者**自身没有数据**，它通过OLE DB接口封装服务，从下层获取数据并向上层提供数据，具有提供者和消费者双重身份。
 - （2）**数据提供者**。数据提供者**自己拥有数据**并通过接口形成表格形式的数据。



- **OLE DB的编程模型**

数据管理任务必须由消费者访问数据，由提供者发布数据。

OLE DB编程模型分类：

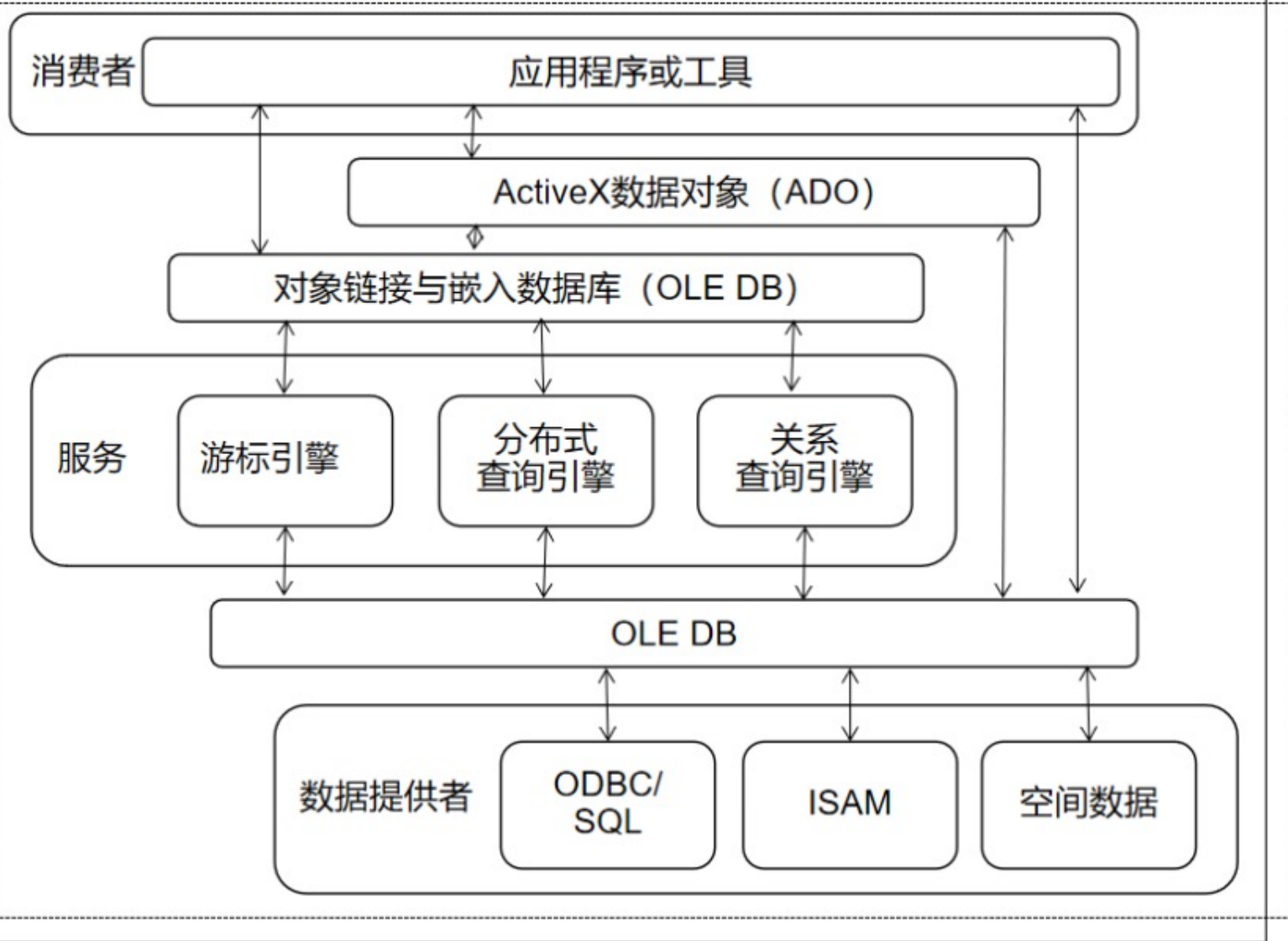
- **Rowset编程模型：**假定数据源中的数据比较规范，提供者以行集形式发布数据。
- **Binder编程模型：**提供者不提供标准表格数据，OLE DB采用Binder编程模型将URL和一个OLE DB对象相关联或绑定，并在必要时创建层次结构对象。



数据库编程



统一数据访问架构 (UDA)



微软事务
服务器



8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

8.5 OLE DB

8.6 JDBC编程

8.7 小结



- **JDBC : Java DataBase Connection**

JDBC是一组Java版的应用程序接口API，提供了Java应用程序与数据库服务器的连接和通讯能力。

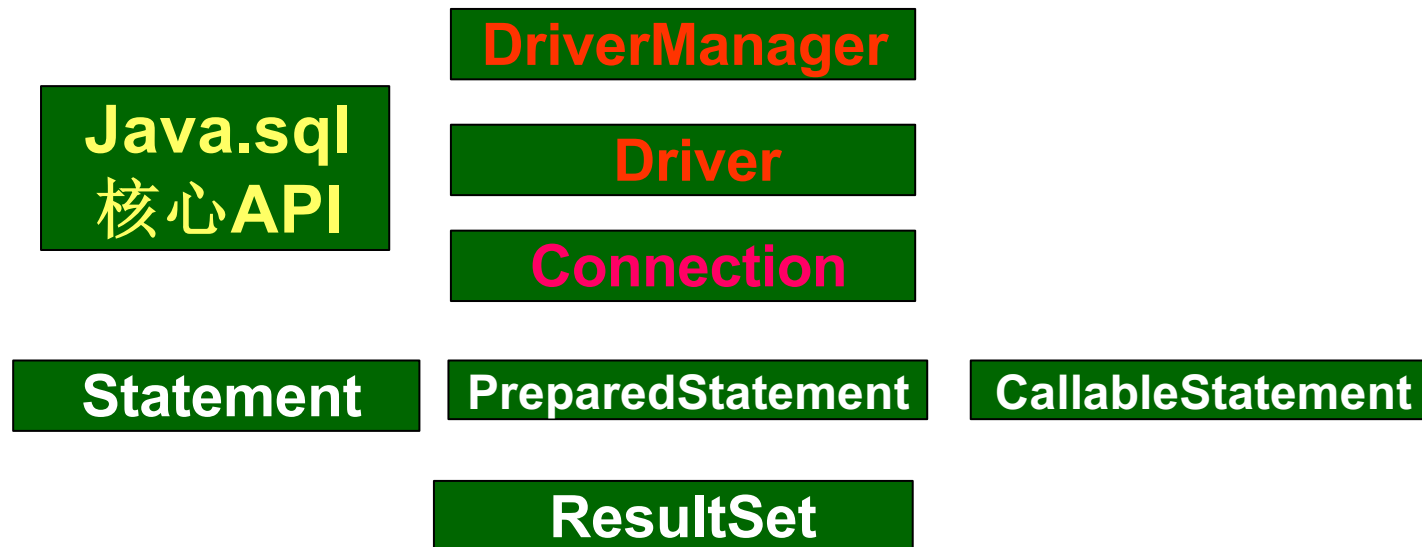
- **(1) JDBC编程是面向对象的接口标准，一般由具体数据库厂商提供。**
- **(2) JDBC编程与ODBC类似，为JAVA提供统一、无缝地操作各种数据库的接口**



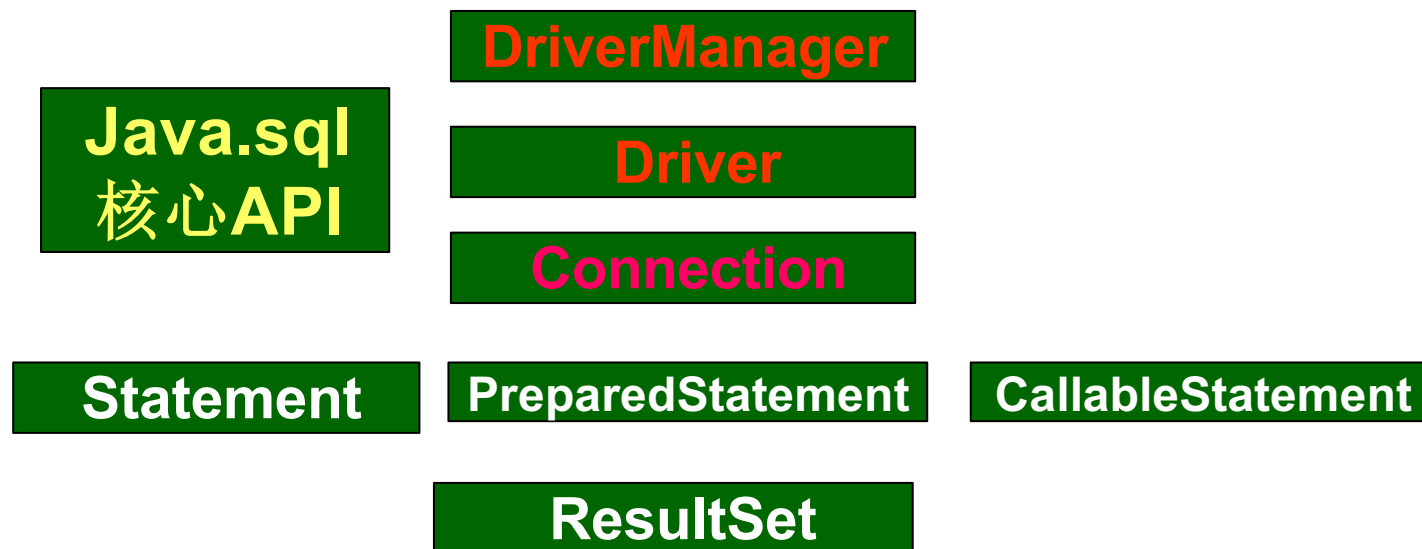
- **JDBC API**

- **JDBC API 分成两个程序包：**

- ✓ **Java.sql 核心API --J2SE(Java2标准版)的一部分。使用 `java.sql.DriverManager`类、`java.sql.Driver`和 `java.sql.Connection`接口连接到数据库**
 - ✓ **Javax.sql 可选扩展API--J2EE(Java2企业版)的一部分。包含了基于JNDI(Java Naming and Directory Interface, Java命名和目录接口)的资源，以及管理**连接池**、**分布式事务**等，使用 `DataSource`接口连接到数据库。**



- **java.sql.DriverManager**——处理驱动的调入并且对产生新数据库连接提供支持
- **Java.sql.Driver**——通过驱动进行数据库访问，连接到数据库的应用程序必须具备该数据库的特定驱动。
- **java.sql.Connection**——代表对特定数据库的连接。
- **Try {...} Catch {...}** ——异常捕获及其处理



- **java.sql.Statement**——对特定的数据库执行SQL语句(静态SQL语句，不支持参数输入)
- **java.sql.PreparedStatement** —— 用于执行预编译的SQL语句（设置SQL参数）
java.sql.CallableStatement ——用于执行对数据库内嵌过程的调用。（可以调用存储过程）
- **java.sql.ResultSet**——从当前执行的SQL语句中返回结果数据。



- 概念性的基本过程

打开一个连接；创建“Statement”对象，并设置查询语句；使用Statement对象执行查询，发送查询给数据库服务器和返回结果给应用程序；处理错误的例外机制

- 具体实施过程

- 传递一个Driver给DriverManager，加载数据库驱动。
- `Class.forName()`
- 通过URL得到一个Connection对象, 建立数据库连接
- `DriverManager.getConnection(sDBUrl)`
- `DriverManager.getConnection(sDBUrl,sDBUserID,sDBPassword)`
- 接着创建一个Statement对象(PreparedStatement或CallableStatement)，用来查询或者修改数据库。
- `Statement stmt=con.createStatement()`
- 查询返回一个ResultSet。
- `ResultSet rs=stmt.executeQuery(sSQL)`



8.1 嵌入式SQL

8.2 过程化SQL

8.3 存储过程和函数

8.4 ODBC编程

8.5 OLE DB

8.6 JDBC编程

8.7 小结



本章讲解了如何使用编程方法对数据库进行操纵的技术

- 1、嵌入式SQL把SQL语言嵌入到高级语言中
- 2、SQL与主语言具有不同的数据处理方式
- 3、过程化SQL编写存储过程和自定义函数
- 4、ODBC工作原理和工作流程
- 5、OLE DB和JDBC的使用